KUB Messenger

Trusted Access for Cyber Proposal

Final defensive security review for a near-production messenger application

Applicant	Maxim Kozlov
Role / Title	Independent Founder and Solo Developer
Email	seraltis13@gmail.com
Company / University	N/A - independent individual project
Supporting professional link	GitHub: https://github.com/ALTIS13
Prepared for	OpenAI Trusted Access for Cyber

Note: This proposal contains no secrets, credentials, private tokens, service-role keys, confidential datasets, or personal user data. It is intended as a public-safe proposal document that can be uploaded to the project repository and linked from the application form.

Project Title

KUB Messenger Security Hardening with Trusted Access for Cyber

One Descriptive Sentence

A near-production messenger application built by a solo developer seeking Trusted Access for Cyber through ChatGPT and Codex to complete a final defensive security review of its codebase, database, deployment pipeline, and AI-assisted development workflow before production release.

Project Proposal

I am building KUB Messenger, a near-production communication application, as an independent solo developer. The project includes user accounts, profiles, role-based access, dynamic permissions, chats, groups, attachments, push notifications, phone verification, recurring tasks, location-based task routing, administrative controls, and production deployment workflows.

The application is already in a late pre-production stage and is planned to enter its initial working production stage in May 2026. A large part of the codebase is developed, refactored, reviewed, and tested with OpenAI Codex. This creates a strong productivity advantage, but it also makes it important to establish a safer AI-assisted development workflow before production launch.

I am requesting access to Trusted Access for Cyber capabilities for legitimate, defensive, and authorized security work on my own application, repository, Supabase database, Edge Functions, deployment configuration, and supporting infrastructure. I am not requesting access for offensive operations, unauthorized testing, third-party exploitation, malware development, credential theft, persistence, evasion, or any activity outside OpenAI policy.

Primary security goals

- Perform a final defensive review of the application codebase before production release.
- Review Supabase Row Level Security policies, RPC functions, database migrations, generated types, and privilege boundaries.
- Verify that role-based and location-scoped permissions are safe, understandable, and consistent across user profiles, admin users, locations, tasks, invites, messaging, and related UI areas.

- Review storage and media access so private or group chat media is not exposed through public or overly broad policies.
- Review push notification, service worker, PWA, phone verification, auth callback, and password recovery flows for safe behavior and sensitive-error redaction.
- Strengthen Codex usage with sandboxed workspaces, approval-based changes, secure migration review, secret-handling checks, CI/CD validation, and defensive regression testing.
- Ensure production monitoring and error reporting are useful while avoiding exposure of message content, tokens, passwords, private media URLs, or other sensitive data.

Current project status

The project already has significant defensive QA and production-readiness work in place. Existing QA results show work across Supabase RLS/RPC checks, generated database types, Playwright browser QA, multi-viewport testing, role and permission hardening, recurring-task scheduler verification, push and phone flow regression coverage, long-session realtime stability checks, and safe credential-handling practices. Trusted Access would be used to complete a final structured defensive review before release.

What Problem Are You Trying to Solve?

I am trying to solve the security gap faced by a solo developer building a near-production messenger application without a dedicated application security team. The application handles user accounts, private and group communication, attachments, permissions, administrative actions, push notifications, recurring tasks, and database-backed access control. Mistakes in authorization, RLS policies, secrets handling, storage access, migrations, or deployment configuration could create serious privacy and security risks.

Because much of the code is written and reviewed through Codex, I also need a safe AI-assisted development workflow that prevents insecure generated code, accidental exposure of secrets, unsafe migrations, broad database permissions, and incomplete security review. The core problem is how to launch and maintain a secure communication application with limited human resources while still applying strong defensive security practices to the codebase, database, infrastructure, and release process.

Requested Funding / API Credits / Resources Needed

I am not requesting API credits or direct financial funding at this stage.

I am requesting access to Trusted Access for Cyber capabilities that can be used through ChatGPT and Codex for authorized defensive security work on my own application, codebase, database, and infrastructure.

Requested resources

- Access to cybersecurity-focused tools and model capabilities inside ChatGPT and Codex for defensive security review.
- Support for secure AI-assisted development workflows, including code review, migration review, RLS and database policy review, dependency review, secrets-handling checks, CI/CD hardening, and production-readiness analysis.
- Ability to use Codex safely as part of my development process, including sandboxed review, approval-based changes, secure refactoring, and defensive testing of my own repository.
- Guidance or tooling that helps identify and remediate vulnerabilities in my own application before and after production launch.

I do not need API credits if Trusted Access allows me to use the relevant cybersecurity tools directly through Codex and ChatGPT. My goal is to improve the security of my own messenger application, not to perform offensive testing or testing of third-party systems.

Project Timeline

May 5-20, 2026 - Production-readiness and security baseline
Initial production QA and security review were completed for the live test environment. Supabase RLS, RPC-based task mutations, realtime behavior, auth callback behavior, responsive UI, task privacy/assignment, chat privacy, message consistency, recurring tasks, dynamic roles, and deployment assumptions were reviewed and progressively hardened.

May 20-24, 2026 - Final Trusted Access / Codex defensive review
Use Trusted Access for Cyber through ChatGPT and Codex to review the application codebase, Supabase RLS policies, RPC functions, database migrations, Edge Functions, storage policies, service worker behavior, secrets handling, deployment configuration, dependency risks, CI/CD assumptions, and production monitoring.

May 24-27, 2026 - Pre-launch remediation
Apply high-priority fixes from the review, especially around authorization boundaries, service-role isolation, frontend/backend trust separation, storage/media access, push notification safety, phone verification behavior, sensitive error redaction, and migration safety.

May 27-31, 2026 - Final regression QA and production cutover
Run full browser QA across desktop and mobile viewports, RLS/RPC smoke tests, multi-account role visibility checks, recurring scheduler checks, push/phone regression tests, monitoring redaction checks, build/typecheck/lint validation, and safe production cutover.

End of May 2026 - Initial working production stage
Release KUB Messenger into its initial working production stage after high-priority security issues are resolved and final regression checks pass.

June 2026 - Post-launch defensive monitoring and hardening
Continue authorized defensive review, regression testing, migration review, monitoring review, and production hardening as real usage begins.

Defensive Use Boundaries

All intended use would be limited to systems, repositories, databases, accounts, networks, and infrastructure that I own, operate, or am explicitly authorized to test or analyze. Trusted Access would not be made available to external customers or third parties.

- No unauthorized testing of third-party systems.
- No malware development, credential theft, persistence, evasion, phishing, or abuse automation.
- No attempts to compromise systems outside my own project and infrastructure.
- No submission of private production user data, confidential datasets, tokens, credentials, or service-role secrets into public documents or repository files.
- All work is intended for defensive security testing, vulnerability identification, remediation, secure software development, and production readiness of my own application.

Expected Outcome

The expected outcome is a safer production launch of KUB Messenger and a repeatable defensive security workflow that can be maintained by one developer. Trusted Access through ChatGPT and Codex would help me perform structured security review, identify risky code or configuration before release, remediate issues faster, and keep the application safer as it grows after launch.

Supporting Link

GitHub profile and project-work verification: https://github.com/ALTIS13